

Evaluación: Unidad II

Apellidos y Nombres: Kenny Leonel Ccora Quispe

1. ¿Cuáles son las principales diferencias entre la optimización de funciones convexas y no convexas, y cómo influyen en la elección de métodos?

Aquí las principales diferencias:

Funciones Convexas:

- **Característica principal:** Cualquier mínimo local es también mínimo global
- **Ventajas:** Garantía de convergencia al óptimo global
- **Métodos efectivos:** Método de Newton, BFGS, métodos de punto interior
- **Aplicación en ML:** SVM, regresión lineal con regularización

Funciones No Convexas:

- **Problema principal:** Múltiples mínimos locales
- **Desafíos:** Riesgo de quedar atrapados en soluciones subóptimas
- **Métodos necesarios:** Algoritmos evolutivos, PSO, algoritmos genéticos
- **Estrategias:** Múltiples inicializaciones, métodos estocásticos

2. ¿Cuándo resulta más efectivo usar variantes del descenso de gradiente y cuáles son los factores clave en su selección?

Los criterios de optimalidad permiten determinar si una solución obtenida por un algoritmo es efectivamente óptima (ya sea local o global). En métodos de optimización, su aplicación depende del tipo de problema:

SGD básico:

- **Uso efectivo:** Datasets grandes
- **Ventaja:** Menor costo computacional por iteración
- **Desventaja:** Convergencia ruidosa

SGD con Momentum:

- **Uso efectivo:** Superficies con barrancos o mal condicionamiento
- **Ventaja:** Acelera convergencia, reduce oscilaciones

Algoritmos adaptativos (Adam, RMSprop, AdaGrad):

- **Uso efectivo:** Mayoría de problemas de ML
- **Ventaja:** Ajuste automático del learning rate por parámetro

Factores clave de selección:

- Tamaño del dataset
- Recursos computacionales disponibles
- Naturaleza de la función objetivo
- Necesidad de convergencia estable vs velocidad

3. ¿Cómo se aplican los criterios de optimalidad en un método de optimización?

Los criterios de optimalidad permiten saber si una solución alcanzada por un algoritmo es suficientemente buena (óptima). Su aplicación varía según el tipo de problema, pero en términos generales se sigue este enfoque:

Sin restricciones:

- **Condición necesaria:** $\nabla f(x^*) = 0$
- **Condición suficiente:** Hessiano positivo definido
- **Implementación:** Criterio de parada $\|\nabla f(x)\| < \epsilon$

Con restricciones de igualdad:

- **Método:** Multiplicadores de Lagrange
- **Condiciones:** $\nabla L(x^*, \lambda^*) = 0$

Con restricciones de desigualdad:

- **Condiciones KKT:** Gradiente del Lagrangiano = 0
- **Holguras complementarias:** $\lambda_i g_i(x^*) = 0$
- **Factibilidad:** $g_i(x^*) \leq 0, \lambda_i \geq 0$

4. ¿De qué manera facilitan las herramientas de optimización automática (p.ej., Optuna) la búsqueda de configuraciones óptimas en aprendizaje automático?

Optuna y otras bibliotecas de optimización automática simplifican la tarea de encontrar hiperparámetros óptimos en modelos de machine learning, algo que manualmente puede ser lento y poco efectivo. Aquí cómo se hace:

Automatización inteligente:

- **Algoritmos:** TPE (Tree-structured Parzen Estimator)
- **Aprendizaje:** Usa evaluaciones anteriores para proponer mejores configuraciones
- **Eficiencia:** Pruning automático elimina trials no prometedores

Beneficios prácticos:

- **Paralelización:** Optimización distribuida
- **Integración:** Compatible con frameworks de ML (scikit-learn, TensorFlow)
- **Ahorro de tiempo:** Evita búsqueda manual exhaustiva

5. ¿Cómo influye la selección de estrategias de regularización en el desempeño y la generalización de un modelo?

La regularización controla la complejidad del modelo para evitar que se ajuste demasiado a los datos de entrenamiento (overfitting). Aquí más detalles:

L1 (Lasso):

- **Efecto:** Selección de características (sparsity)
- **Uso:** Cuando queremos pocas características relevantes
- **Impacto:** Reduce overfitting eliminando características irrelevantes

L2 (Ridge):

- **Efecto:** Reduce magnitud de pesos sin eliminarlos
- **Uso:** Cuando todas las características son importantes
- **Impacto:** Mejora generalización suavizando el modelo

Elastic Net:

- **Combinación:** $L1 + L2$
- **Uso:** Grupos de características correlacionadas

Impacto en desempeño:

- **Bias-variance tradeoff:** Aumenta sesgo, reduce varianza
- **Generalización:** Mejora rendimiento en datos no vistos
- **Selección λ :** Validación cruzada para parámetro óptimo

6. ¿Cómo se pueden integrar rutinas de validación y corrección de código en Optuna para mejorar la calidad y eficacia del proceso de optimización de hiperparámetros?

Incorporar rutinas de validación y corrección dentro del proceso de optimización con Optuna permite asegurar que cada prueba sea confiable, interpretable y reproducible. Aquí más detalles:

Rutinas de validación:

- **Cross-validation:** Dentro de la función objetivo
- **Pruning:** `trial.report()` y `trial.should_prune()`
- **Early stopping:** Basado en métricas de validación

Corrección y monitoreo:

- **Logging:** Registro automático de todos los trials
- **Callbacks:** Guardar mejores modelos automáticamente
- **Manejo de errores:** Try-catch para trials fallidos
- **Visualización:** Optuna-dashboard para monitoreo en tiempo real

7. ¿Cómo podemos utilizar las métricas provistas por scikit-learn para evaluar un modelo de clasificación, y qué información adicional brindan además de la exactitud? Completa y Responde:

Scikit-learn ofrece varias métricas que permiten evaluar modelos de clasificación con más profundidad que solo la exactitud (accuracy). Estas métricas revelan cómo se com-

porta el modelo frente a distintas clases, desequilibrios y errores.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

y_true = [0, 1, 2, 2, 2]
y_pred = [0, 2, 2, 2, 1]

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred, average='macro'))
print("Recall:", recall_score(y_true, y_pred, average='macro'))
```

Figura 1: Ejemplo de uso de métricas en scikit-learn

Resultado:

- Accuracy: 0.6
- Precision: 0.5555555555555555
- Recall: 0.5555555555555555

Información adicional:

Precision:

- **Mide:** De las predicciones positivas, cuántas fueron correctas
- **Fórmula:** $TP / (TP + FP)$
- **Importante:** Cuando falsos positivos son costosos

Recall:

- **Mide:** De los casos positivos reales, cuántos fueron detectados
- **Fórmula:** $TP / (TP + FN)$
- **Importante:** Cuando falsos negativos son costosos

Ventajas sobre accuracy:

- **Clases desbalanceadas:** Accuracy puede ser engañosa
- **Análisis por clase:** Identifica qué clases se predicen mal
- **F1-score:** Combina precision y recall en una métrica
- **Matriz de confusión:** Muestra errores específicos entre clases